

Московский физико-технический институт
Физтех-школа прикладной математики и информатики

А л г о р и т м ы и с т р у к т у р ы д а н н ы х
I СЕМЕСТР

Лектор: *Степанов Илья Данилович*



Автор: *Еремей Дмитриенко*

Telegram: t.me/dm1trienko

VK: vk.com/dm1trienko

осень 2025

Содержание

Лекция 1. Модель вычислений и базовые алгоритмы	2
Постановка задач и структура решения	2
Элементарные действия в RAM-модели	2
Пример: поиск минимума	2
Асимптотические обозначения	3
Бинарный поиск	3
Техника префиксных сумм	4
Лекция 2. Сортировки (часть I)	6
Две постановки задачи	6
Эквивалентность постановок	6
Нижняя оценка для сортировок сравнением	6
Сортировка слиянием	7
Быстрая сортировка (Quick Sort)	7
Среднее время работы Quick Sort	10
Лекция 3. Сортировки (часть II)	12
Порядковые статистики и Quick Select	12
Детерминированный выбор медианы (Median of Medians)	13
Quick Sort с детерминированным pivot	15
Сортировка подсчётом	15
Сортировка пар	16
Radix Sort (LSD)	16
Лекция 4. Списки, стеки, очереди	17
Что такое структура данных	17
Связные списки	17
Двусвязные списки и базовые операции	17
Стек	17
Монотонный стек: ближайший больший элемент слева	18
Проверка правильности скобочной последовательности	18
Вычисление выражения в обратной польской записи	18
Очередь и амортизированный анализ	18
Лекция 5. Кучи: расширенный функционал	24
Операция <code>decrease_key</code>	24

Удаление произвольного элемента (по указателю)	24
Процедура <code>sift_down</code>	25
Биномиальные кучи	25
Операции и сложность биномиальной кучи	26
Фибоначчиевы кучи	27
Уменьшение ключа и «каскадные» обрезки	28
Амортизированный анализ фибоначчиевой кучи	28
Стек с минимумом	29
Очередь с минимумом	30
Амортизированный анализ очереди с минимумом	30
Лекция 6. Удаление из кучи	31
Новая операция <code>erase(x)</code>	31
Практические замечания	31
Биномиальные кучи	32
Операции	32
Сложность	33
Фибоначчиевы кучи	33
Амортизированное время	33

Лекция 1. Модель вычислений и базовые алгоритмы

Постановка задач и структура решения

В течение курса мы будем систематически решать алгоритмические задачи. Для каждой задачи полезно фиксировать единую структуру решения:

1. **Условие.** Чёткое описание входных данных и требуемого результата.
2. **Алгоритм.** Пошаговый способ, преобразующий вход в выход. Мы будем давать как словесные, так и псевдокодированные описания.
3. **Корректность.** Доказательство того, что алгоритм всегда выдаёт правильный ответ. Чаще всего — индукцией, инвариантами или редукцией к уже известным утверждениям.
4. **Асимптотика времени.** Оценка количества элементарных действий в зависимости от размера входа n .
5. **Число элементарных операций.** Явный подсчёт (или хотя бы оценка сверху) арифметических действий, сравнений, обращений к памяти и т. д.

Элементарные действия в RAM-модели

Работа алгоритма анализируется в рамках модели *Random Access Machine*. Мы считаем элементарными:

- ▷ арифметические операции $+$, $-$, $\cdot$, $/$, остаток от деления;
- ▷ сравнения вида $a < b$, $a = b$;
- ▷ булевы операции ($\wedge$, $\vee$, $\oplus$, побитовые сдвиги и т. д.);
- ▷ обращение к произвольной ячейке памяти по индексу за $O(1)$ (операция случайного доступа, Random Access);
- ▷ условные переходы, реализуемые конструкцией `if {...} else {...}`.

Пример: поиск минимума

Пусть дан массив $A = (a_0, a_1, \dots, a_{n-1})$. Требуется вывести минимальный элемент. Последовательность действий:

1. Считать n и элементы массива.
2. Положить $x := a_0$.
3. Для каждого $i = 1, \dots, n - 1$:
 - (a) если $a_i < x$, то заменить $x := a_i$.
4. Вывести x .

Подсчёт показывает, что выполняется $6n + 4$ элементарных действий (инициализация, тело цикла, проверка и т. д.).

```

1 int find_min(const vector<int>& a) {
2     int x = a[0];                // current minimum
3     for (size_t i = 1; i < a.size(); ++i) {
4         if (a[i] < x) {
5             x = a[i];
6         }
7     }
8     return x;
9 }

```

Функция поиска минимума в массиве

Асимптотические обозначения

Определение 0.1. Для функций $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ говорят, что $f(n) = O(g(n))$, если существуют константы $C > 0$ и $N \in \mathbb{N}$ такие, что при всех $n \geq N$ выполнено неравенство $f(n) \leq C \cdot g(n)$.

Определение 0.2. Аналогично, $f(n) = \Omega(g(n))$, если существуют $C > 0$ и N со свойством $f(n) \geq C \cdot g(n)$ при всех $n \geq N$.

Определение 0.3. Говорят, что $f(n) = \Theta(g(n))$, если одновременно $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$. В этом случае функции эквивалентны с точностью до постоянного множителя.

Пример. Пусть $f(n) = 6n + 4$ и $g(n) = n$. Тогда $f(n) = O(g(n))$ (достаточно взять $C = 10$, $N = 4$) и $f(n) = \Omega(g(n))$ (подходят $C = 6/7$, $N = 1$). Следовательно, $f(n) = \Theta(g(n))$.

Замечание. Если $f(n) = \Theta(g(n))$, то $g(n) = \Theta(f(n))$. Более того, из $f(n) = O(g(n))$ сразу следует $g(n) = \Omega(f(n))$, и наоборот.

Утверждение 0.1. Пусть $f(n) \leq C \cdot g(n)$ для всех $n \geq N$. Тогда можно выбрать константу $\hat{C} = \max\left(C, \frac{f(1)}{g(1)}, \dots, \frac{f(N)}{g(N)}\right)$ и утверждать $f(n) \leq \hat{C} \cdot g(n)$ при всех $n \geq 1$.

Доказательство. ► Для $n \geq N$ неравенство выполнено по определению. Для $n < N$ слева стоит одна из подразумеваемых дробей, поэтому выбор $\hat{C}$ обеспечивает единую оценку на всей области. $\square$

Бинарный поиск

Пусть A — отсортированный по возрастанию массив. Задача: выяснить, встречается ли число x среди элементов.

Задача. Найдите индекс i такой, что $a_i = x$, либо установите, что элемента нет. Предполагается, что элементы массива отсортированы: $a_0 \leq a_1 \leq \dots \leq a_{n-1}$.

Поддерживаем полуинтервал $(\ell, r]$ (изначально $\ell = -1$, $r = n$) с инвариантом: если x присутствует, то он лежит в этом полуинтервале. На каждом шаге вычисляем середину $m = \lfloor (\ell + r)/2 \rfloor$ и сравниваем a_m с x :

▷ если $x > a_m$, сдвигаем левую границу $\ell := m$;

▷ иначе правую границу $r := m$.

Цикл продолжается, пока $r - \ell > 1$. После выхода проверяем r : если $r < n$ и $a_r = x$, элемент найден.

```

1 int binary_search(const vector<int>& a, int x) {
2     int l = -1, r = (int)a.size();
3     while (r - l > 1) {
4         int m = (l + r) / 2;
5         if (a[m] < x) {
6             l = m;
7         } else {
8             r = m;
9         }
10    }
11    if (r < (int)a.size() && a[r] == x) {
12        return r;
13    }
14    return -1; // element not found
15 }
```

Реализация бинарного поиска

Техника префиксных сумм

Рассмотрим множество онлайн-запросов к массиву A : для пар $0 \leq \ell \leq r < n$ требуется вычислять сумму $a_\ell + a_{\ell+1} + \dots + a_r$.

▷ **Предобработка.** Построим массив префиксных сумм p : $p[0] = a_0$, $p[i] = p[i-1] + a_i$ для $1 \leq i < n$. Предобработка занимает $O(n)$ времени.

▷ **Ответ на запрос.** Для $0 < \ell \leq r$ значение равно $p[r] - p[\ell - 1]$; для $\ell = 0$ используем $p[r]$. Стоимость одного запроса — $O(1)$, общая сложность обработки q запросов после предобработки составляет $O(n + q)$.

Замечание. Для преобразования логарифмов полезно помнить формулу смены основания: $\log_a n = \frac{\log_b n}{\log_b a}$. Она часто используется при оценке сложности алгоритмов.

```
1 vector<long long> build_prefix(const vector<int>& a) {  
2     vector<long long> pref(a.size());  
3     pref[0] = a[0];  
4     for (size_t i = 1; i < a.size(); ++i) {  
5         pref[i] = pref[i - 1] + a[i];  
6     }  
7     return pref;  
8 }  
9  
10 long long range_sum(const vector<long long>& pref, size_t l, size_t r) {  
11     return (l == 0 ? pref[r] : pref[r] - pref[l - 1]);  
12 }
```

Префиксные суммы и ответ на запрос диапазона

Лекция 2. Сортировки (часть I)

Две постановки задачи

Пусть дан массив $A = (a_1, \dots, a_n)$.

1. **Задача 1.** Найти отсортированный массив $B = (b_1, \dots, b_n)$, удовлетворяющий $b_1 \leq b_2 \leq \dots \leq b_n$ и совпадающий с A как мультимножество.
2. **Задача 2.** Найти перестановку $\sigma \in S_n$, такую что $a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)}$.

Замечание. Первая формулировка отвечает на вопрос «каков итоговый отсортированный список значений», вторая — «в каком порядке следует переставлять исходные элементы». Обе постановки эквивалентны: по найденной перестановке легко построить отсортированный массив и наоборот.

Утверждение 0.2. Пусть $T(n) = \Omega(n)$. Если существует алгоритм, решающий одну из двух задач сортировки за $O(T(n))$, то можно построить алгоритм, решающий и вторую задачу за $O(T(n))$.

Доказательство. Для перехода $1 \Rightarrow 2$ отсортируем пары (a_i, i) лексикографически и прочитаем перестановку из вторых координат. Для перехода $2 \Rightarrow 1$ достаточно выписать элементы в порядке $\sigma(1), \dots, \sigma(n)$. Оба преобразования занимают $O(n)$, что поглощается $O(T(n))$. $\square$

Эквивалентность постановок

Чтобы показать переход $1 \Rightarrow 2$, достаточно отсортировать пары (a_i, i) лексикографически. Если $(x_1, y_1) < (x_2, y_2)$, когда либо $x_1 < x_2$, либо $x_1 = x_2$ и $y_1 < y_2$, то порядок полученных пар определяет требуемую перестановку. В обратную сторону (из перестановки к отсортированному массиву) достаточно расставить элементы на новые позиции: на место i поставить $a_{\sigma(i)}$.

Нижняя оценка для сортировок сравнением

Теорема 0.1. Любой алгоритм сортировки, использующий только сравнения элементов, в худшем случае выполняет $\Omega(n \log n)$ сравнений.

Идея доказательства. ► Рассмотрим решающее дерево: вершины соответствуют сравнениям вида $a_i \leq a_j$, а листья — перестановкам, которые могут быть результатом сортировки. Если входы попарно различны, каждый лист однозначно задаёт одну перестановку. Следовательно, число листьев не меньше $n!$. Глубина дерева определяет максимальное число сравнений. Для двоичного дерева с L листьями глубина не меньше $\log_2 L$, значит, требуется $\log_2(n!) = \Omega(n \log n)$ сравнений. $\square$

Лемма 0.1. Если алгоритм выполняет не более k сравнений, то количество возможных протоколов выполнения (последовательностей ответов «меньше/больше») не превосходит $\sum_{\ell=0}^k 2^\ell \leq 2^{k+1}$.

Доказательство. Для фиксированной длины ℓ каждое сравнение имеет два исхода, следовательно, существует ровно 2^ℓ последовательностей. Суммируя по $\ell \leq k$ получаем геометрическую прогрессию, ограниченную 2^{k+1} . $\square$

Лемма 0.2. *Выполняются соотношения $\log_2(n!) = O(n \log n)$ и $\log_2(n!) = \Omega(n \log n)$.*

Доказательство. Верхняя оценка: $n! = 1 \cdot 2 \cdot \dots \cdot n \leq n^n$, значит $\log_2(n!) \leq n \log_2 n$.

Нижняя оценка: возьмём произведение последних $\lfloor n/2 \rfloor$ множителей. Каждый из них не меньше $n/2$, поэтому

$$n! \geq \left(\frac{n}{2}\right)^{\lfloor n/2 \rfloor}, \quad \log_2(n!) \geq \frac{n}{2}(\log_2 n - 1) = \Omega(n \log n).$$

□

Продолжение доказательства теоремы о нижней границе. ► Пусть k — максимальная глубина решающего дерева. По лемме число протоколов работы (а значит и листьев) не превосходит 2^{k+1} . С другой стороны, каждый лист задаёт уникальную перестановку, поэтому $L \geq n!$. Следовательно, $2^{k+1} \geq n!$ и $k + 1 \geq \log_2(n!) = \Theta(n \log n)$. □

Сортировка слиянием

1. Разделить массив A на левую и правую половины L и R (размеры отличаются не более чем на 1).
2. Рекурсивно отсортировать массивы L и R .
3. Слить отсортированные части в итоговый массив, выбирая на каждом шаге минимальный из текущих элементов.

Функция слияния работает за $O(|L| + |R|)$, а вся сортировка удовлетворяет рекурренте $T(n) = 2T(\lfloor n/2 \rfloor) + O(n)$. Из анализа рекуррентности (мастер-теорема, математическая индукция или рисунок-рекурсия) следует $T(n) = O(n \log n)$.

Рекурсия раскладывает массив на половины, пока размер не станет единичным. На пути обратно результаты сливаются функцией MERGE; суммарная работа на каждом уровне дерева рекурсии равна $O(n)$, а глубина — $O(\log n)$.

Пример. Merge Sort позволяет также подсчитывать число инверсий: при слиянии достаточно учитывать, сколько элементов остаётся в левой части при выборе элемента из правой.

Быстрая сортировка (Quick Sort)

1. Случайно выбрать опорный элемент x (pivot) из массива A .
2. Разделить элементы на три группы: $L = \{a_i < x\}$, $M = \{a_i = x\}$ и $R = \{a_i > x\}$.
3. Рекурсивно отсортировать L и R , блок M остаётся неизменным.
4. Сформировать ответ конкатенацией L, M, R .

Основная работа сосредоточена в процедуре PARTITION, которая должна выполнить разбиение за линейное время и по возможности без дополнительных массивов. Классическая реализация использует два указателя, двигающихся навстречу друг другу.

```
1 vector<int> merge(const vector<int>& left, const vector<int>& right) {
2     vector<int> merged;
3     merged.reserve(left.size() + right.size());
4     size_t i = 0, j = 0;
5     while (i < left.size() || j < right.size()) {
6         if (j == right.size() || (i < left.size() && left[i] <= right[j])) {
7             merged.push_back(left[i++]);
8         } else {
9             merged.push_back(right[j++]);
10        }
11    }
12    return merged;
13 }
14
15 vector<int> merge_sort(const vector<int>& a) {
16     if (a.size() <= 1) {
17         return a;
18     }
19     size_t mid = a.size() / 2;
20     vector<int> left(a.begin(), a.begin() + mid);
21     vector<int> right(a.begin() + mid, a.end());
22     return merge(merge_sort(left), merge_sort(right));
23 }
```

Функции merge и merge_sort

```
1 size_t partition(vector<int>& a, size_t left, size_t right, int pivot) {
2     size_t i = left, j = right;
3     while (i <= j) {
4         while (a[i] < pivot) ++i;
5         while (a[j] > pivot) --j;
6         if (i <= j) {
7             swap(a[i], a[j]);
8             ++i;
9             if (j > 0) --j;
10        }
11    }
12    return i;
13 }
14
15 void quick_sort(vector<int>& a, size_t left, size_t right) {
16     if (left >= right) {
17         return;
18     }
19     int pivot = a[left + rand() % (right - left + 1)];
20     size_t split = partition(a, left, right, pivot);
21     if (split > 0) {
22         quick_sort(a, left, split - 1);
23     }
24     quick_sort(a, split, right);
25 }
```

Алгоритм Quick Sort с in-place partition

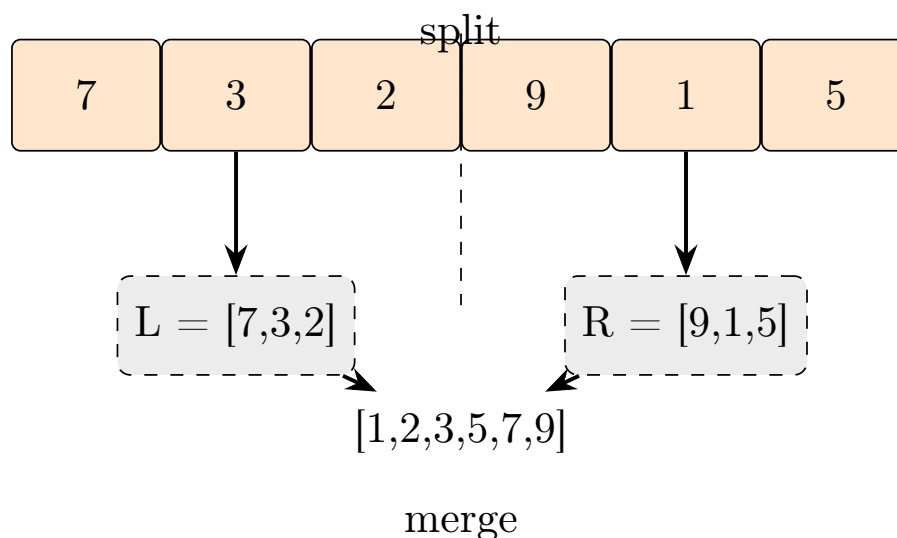


Рис. 1: Рекурсивное разбиение в Merge Sort: на каждом уровне суммарная работа остаётся $O(n)$.

Среднее время работы Quick Sort

Пусть $T(n)$ — случайная величина, равная числу сравнений при сортировке массива из n элементов. После выбора `pivot` левый подмассив содержит k элементов, правый — $n - k - 1$. Затраты на один уровень равны $\Theta(n)$, поэтому

$$\mathbb{E}[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} (\mathbb{E}[T(k)] + \mathbb{E}[T(n - k - 1)]) + O(n),$$

где левый и правый размеры равновероятны. Индукцией по n или методом развёртывания рекурсии получаем $\mathbb{E}[T(n)] = O(n \log n)$. Случайный выбор `pivot` защищает от систематически неудачных входов (например, уже отсортированного массива), на которых детерминированные стратегии вроде «берём первый элемент» работают за $O(n^2)$.

Замечание. На семинаре обсуждается реализация `PARTITION` без выделения памяти и примеры входов, провоцирующих худший случай.

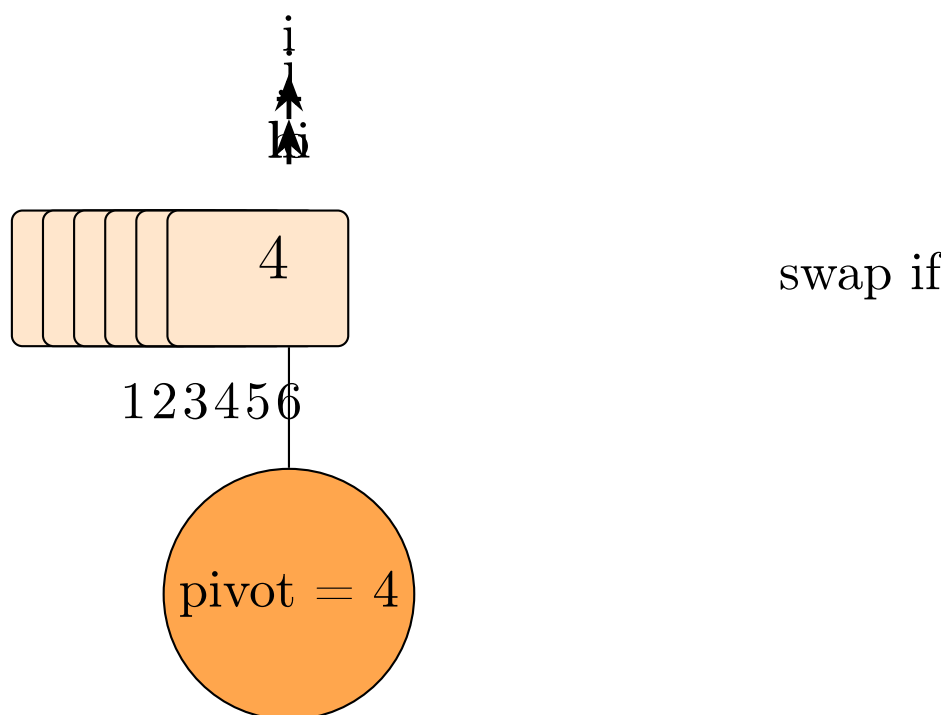


Рис. 2: Разбиение массива вокруг опорного элемента в Quick Sort.

Лекция 3. Сортировки (часть II)

Порядковые статистики и Quick Select

Определение 0.4. k -я порядковая статистика массива A — элемент, который окажется на k -й позиции, если массив отсортировать по возрастанию.

Простейший способ найти статистику — отсортировать весь массив, но это избыточно. Алгоритм Quick Select работает быстрее:

1. Выбрать случайный элемент x (*pivot*).
2. Разбить массив на три части: элементы меньше x , равные x и больше x (операция PARTITION).
3. Определить, в какую часть попадает искомая статистика, и рекурсивно обрабатывать только её.

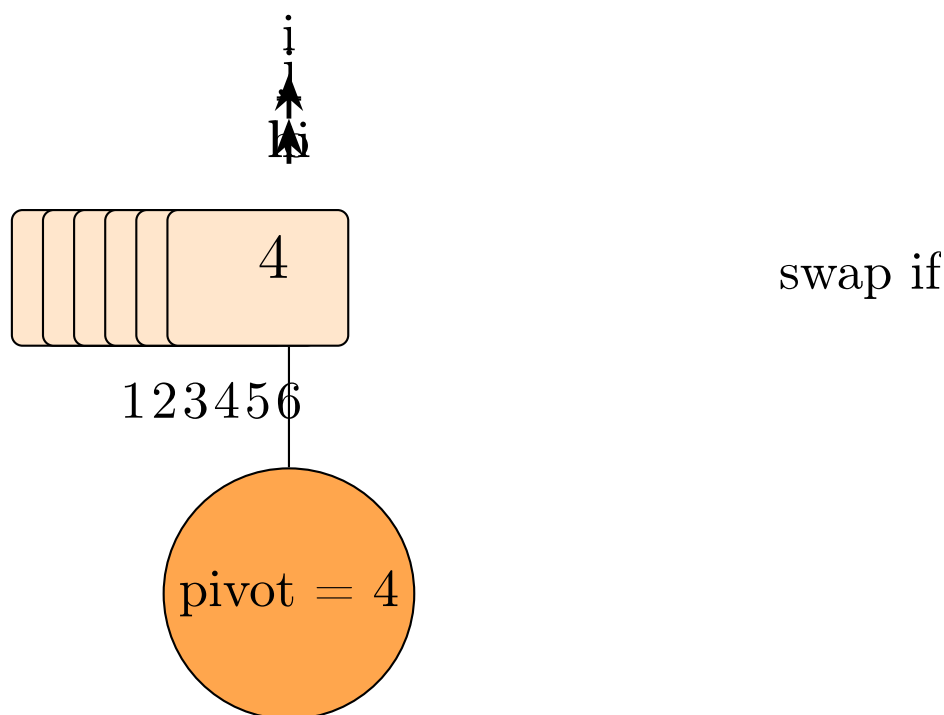


Рис. 3: Разделение массива вокруг pivot: рекурсия идёт только в одну из трёх частей.

Теорема 0.2. При случайном выборе опорного элемента Quick Select имеет среднее время работы $O(n)$.

Идея. ► Вероятность того, что pivot окажется достаточно «хорошим» (разделит массив хотя бы в пропорции 1 : 3), постоянна. В среднем количество элементов в рекурсивном вызове убывает геометрически, а суммарная работа ограничена линейной функцией от n . $\square$

Замечание. Пусть $T(n)$ — случайная величина, равная числу сравнений. Разбив массив на части размера k и $n - k - 1$, получаем рекуррентность

$$\mathbb{E}[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} (\mathbb{E}[T(k)] + \mathbb{E}[T(n - k - 1)]) + O(n),$$

откуда индукцией следует $\mathbb{E}[T(n)] = O(n)$. Худший случай (pivot всегда минимальный или максимальный) даёт $O(n^2)$, поэтому ключевую роль играет случайный выбор опорного элемента. Так работает стандартный алгоритм `nth_element` в C++.

```

1 int quick_select(vector<int> a, int k) {
2     if (a.size() == 1) {
3         return a[0];
4     }
5     int pivot = choose_random(a);           // pick arbitrary array element
6     vector<int> L, M, R;
7     for (int value : a) {
8         if (value < pivot) L.push_back(value);
9         else if (value > pivot) R.push_back(value);
10        else M.push_back(value);
11    }
12    if (k < (int)L.size()) {
13        return quick_select(L, k);
14    }
15    if (k < (int)L.size() + (int)M.size()) {
16        return pivot;
17    }
18    return quick_select(R, k - (int)L.size() - (int)M.size());
19 }

```

Алгоритм Quick Select

Каждый рекурсивный вызов обрабатывает только ту часть, где точно лежит ответ, поэтому глубина рекурсии пропорциональна числу удачных разбиений. Поддерживается инвариант: множество элементов в объединении L, M, R совпадает с исходным массивом, а границы по k задают, в какую часть нужно «нырнуть» дальше.

Детерминированный выбор медианы (Median of Medians)

Алгоритм Блум—Флойда—Пратта—Ривеста—Тарджана позволяет подбирать pivot без случайности:

1. Разбить массив A на блоки по 5 элементов (последний блок может быть короче).
2. Отсортировать каждый блок за константное время и выписать медиану b_i каждого блока.
3. Рекурсивно вычислить медиану массива $B = (b_i)$; обозначим её через x .

4. Использовать x как pivot и продолжить Quick Select только в нужной части.

Блоки размера 5 гарантируют, что в каждом блоке не менее трёх элементов $\geq b_i$ и трёх элементов $\leq b_i$.

Лемма 0.3. Медиана x массива B лежит между $\frac{3n}{10}$ -й и $\frac{7n}{10}$ -й статистиками массива A .

Доказательство. ► По крайней мере половина медиан b_i не меньше x . Каждая такая медиана даёт три элемента исходного массива, не меньше x , поэтому элементов $\geq x$ не менее $3 \cdot \frac{1}{2} \lfloor n/5 \rfloor \geq 3n/10 - 3$. Аналогично для элементов $\leq x$. Для достаточно больших n константа поглощается, и x действительно находится между указанными порядковыми статистиками. $\square$

Отсюда рекуррентность

$$T(n) \leq T(\lceil n/5 \rceil) + T\left(\frac{7n}{10}\right) + cn,$$

которая решается индукцией: взяв $C \geq 10c$, получим $T(n) \leq Cn$ для всех n .

```

1  int deterministic_select(vector<int> a, int k) {
2      if (a.size() <= 5) {
3          sort(a.begin(), a.end());
4          return a[k];
5      }
6      vector<int> medians;
7      for (size_t i = 0; i < a.size(); i += 5) {
8          size_t r = min(i + 5, a.size());
9          vector<int> block(a.begin() + i, a.begin() + r);
10         sort(block.begin(), block.end());
11         medians.push_back(block[block.size() / 2]);
12     }
13     int pivot = deterministic_select(medians, medians.size() / 2);
14     vector<int> L, M, R;
15     for (int value : a) {
16         if (value < pivot)    L.push_back(value);
17         else if (value > pivot) R.push_back(value);
18         else                 M.push_back(value);
19     }
20     if (k < (int)L.size()) {
21         return deterministic_select(L, k);
22     }
23     if (k < (int)L.size() + (int)M.size()) {
24         return pivot;
25     }
26     return deterministic_select(R, k - (int)L.size() - (int)M.size());
27 }
```

Детерминированный выбор медианы (Median of Medians)

Алгоритм всегда выбирает *pivot*, лежащий между $3n/10$ -й и $7n/10$ -й статистиками, поэтому в рекурсивном вызове остаётся не более $0.7n$ элементов. Рекуррентность $T(n) \leq T(n/5) + T(7n/10) + cn$ решается индукцией: $T(n) = O(n)$ без скрытой вероятности.

Quick Sort с детерминированным *pivot*

Если использовать deterministic Quick Select для поиска медианы и каждый раз разбивать массив на две половины одинакового порядка, то сложность Quick Sort становится гарантированно $O(n \log n)$.

При таком выборе *pivot* каждая рекурсивная ветка содержит не более $\lceil n/2 \rceil$ элементов. Следовательно, дерево рекурсии имеет высоту $O(\log n)$ и полностью повторяет структуру Merge Sort, но без дополнительных массивов. На практике медленные константы делают такой подход менее привлекательным; чаще используют рандомизированный Quick Sort, который даёт ожидаемое $O(n \log n)$.

Сортировка подсчётом

Пусть $a_i \in \{0, 1, \dots, k\}$. Заведём массив счетчиков $\text{cnt}[0 \dots k]$, заполним нулями и пройдем по исходному массиву, увеличивая соответствующий счётчик. Затем восстанавливаем отсортированную последовательность, выводя значение x ровно $\text{cnt}[x]$ раз. Время работы $O(n + k)$. Чтобы сделать алгоритм стабильным (важно в радиальных сортировках и при сортировке пар), заменим массив счётчиков на массив префиксных сумм и будем расставлять элементы во второй массив, проходя исходный массив с конца.

Важно, что после шага с префиксами каждая ячейка $\text{cnt}[x]$ хранит позицию, на которую необходимо поставить очередной элемент со значением x . Обход массива A справа налево гарантирует сохранение относительного порядка равных элементов и позволяет восстановить не только отсортированный массив, но и перестановку индексов. Если $k \gg n$, то стоимость построения массива cnt становится доминирующей; в таких случаях сортировку подсчётом применять невыгодно.

```

1 vector<int> counting_sort(const vector<int>& a, int k) {
2     vector<int> cnt(k + 1, 0);
3     for (int value : a) {
4         ++cnt[value];
5     }
6     for (int i = 1; i <= k; ++i) {
7         cnt[i] += cnt[i - 1];           // prefix sums for stability
8     }
9     vector<int> result(a.size());
10    for (int i = (int)a.size() - 1; i >= 0; --i) {
11        int value = a[i];
12        result[--cnt[value]] = value;
13    }
14    return result;
15 }
```

Стабильная сортировка подсчётом

```
1 def counting_permutation(a, k):
2     cnt = [0] * (k + 1)
3     for value in a:
4         cnt[value] += 1
5     for i in range(1, k + 1):
6         cnt[i] += cnt[i - 1]
7     pos = [0] * len(a)
8     for i in range(len(a) - 1, -1, -1):
9         value = a[i]
10        cnt[value] -= 1
11        pos[cnt[value]] = i        # move index i to position cnt[value]
12    return pos
```

Перестановка индексов для сортировки подсчётом

Массив pos описывает перестановку индексов: элемент a_i после сортировки окажется на месте $pos^{-1}(i)$. Такой приём понадобится при лексикографической сортировке пар и в алгоритмах Radix Sort.

Сортировка пар

Для набора пар (x_i, y_i) , где $x_i, y_i \in \{0, \dots, k\}$, выполняем два прохода стабильной сортировки подсчётом:

1. сначала сортируем по вторым координатам y_i ;
2. затем сортируем по первым координатам x_i .

Благодаря стабильности предыдущее упорядочение по y сохраняется внутри каждой группы одинаковых x , что и даёт лексикографический порядок. При необходимости можно комбинировать сортировку подсчётом и хранение перестановки из предыдущего шага.

Radix Sort (LSD)

Пусть каждое число записывается в системе счисления по основанию b и содержит $\ell = \lceil \log_b k \rceil$ разрядов. Выполним ℓ проходов стабильной сортировки по разрядам от младшего к старшему. Выбирая $b = 2^c$, можно добиться времени $O(\log_b k \cdot (n + b))$.

Каждый проход сортирует элементы по одному разряду, начиная с младшего. Благодаря стабильности результаты предыдущих проходов сохраняются, поэтому после обработки самого старшего разряда элементы оказываются упорядочены по всему числу. На практике основание выбирают так, чтобы обрабатывать по несколько бит за раз (например, один байт).

Лекция 4. Списки, стеки, очереди

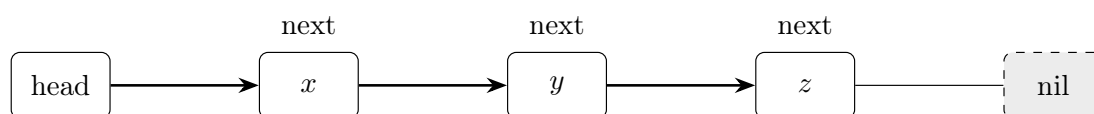
Что такое структура данных

Определение 0.5. Структура данных — способ хранения объектов в памяти компьютера для эффективной поддержки заданного набора операций (онлайн-запросов).

Обычно известен перечень запросов (добавить элемент, удалить, посчитать агрегат и т. д.), и структура подбирается так, чтобы выполнять их быстро.

Связные списки

Линейный список состоит из узлов, каждый из которых хранит значение и указатель на следующий элемент. Головной узел `head` позволяет пройти по всей цепочке. Преимущества:



Синглизвязный список
 $O(1)$ вставка в голову, $O(n)$ доступ по индексу

Рис. 4: Связный список: указатели ведут от `head` к `tail`, последний узел смотрит в $\emptyset$.

вставка и удаление по указателю выполняются за $O(1)$. Недостатки: нет прямого доступа к произвольному элементу (приходится идти от головы), итерация по всем элементам медленнее, чем по массиву.

Двусвязные списки и базовые операции

Если каждому узлу добавить указатель не только на следующего соседа, но и на предыдущего, получим двусвязный список.

- ▷ **Вставка после узла v .** Создаём новый узел u , назначаем ему ссылки на v и $v.next$, затем перенастраиваем соседей. Все изменения занимают $O(1)$.
- ▷ **Удаление узла v .** Перенаправляем указатель из $v.prev$ на $v.next$ и наоборот. После этого освобождаем память под v — снова $O(1)$.
- ▷ **Итераторы.** Хранение указателей позволяет двигаться в обе стороны. Однако позиция « k -й элемент» требует $O(k)$ шагов: приходится последовательно переходить по ссылкам.

Преимущество двусвязного списка перед массивом проявляется там, где важно быстро вставлять и удалять элементы в произвольных местах. Однако для случайного доступа (операции «найди элемент с номером k ») массив остаётся предпочтительным.

Стек

Стек поддерживает операции `push`, `top`, `pop`. Все они выполняются за $O(1)$, в том числе при реализации поверх связного списка.

Монотонный стек: ближайший больший элемент слева

Пусть задан массив a . Требуется для каждого индекса i найти наибольший индекс $j < i$ такой, что $a_j > a_i$ (если такого нет, ответ -1). Поддерживаем стек индексов, образующих строго убывающую последовательность значений. Каждый элемент добавляется и удаля-

```
1 vector<int> previous_greater(const vector<int>& a) {
2     vector<int> ans(a.size(), -1);
3     vector<int> st; // monotonic stack of indices
4     for (size_t i = 0; i < a.size(); ++i) {
5         while (!st.empty() && a[st.back()] <= a[i]) {
6             st.pop_back();
7         }
8         ans[i] = st.empty() ? -1 : (int)st.back();
9         st.push_back((int)i);
10    }
11    return ans;
12 }
```

Монотонный стек для поиска ближайшего большего слева

ется из стека не более одного раза, поэтому суммарная сложность — $O(n)$. Такой шаблон часто называют *монотонным стеком*; его используют в задачах о скользящих окнах, поиске ближайшего большего/меньшего и т. д.

Проверка правильности скобочной последовательности

При чтении строки последовательно добавляем открывающие скобки в стек. Обнаружив закрывающую скобку, сравниваем её с вершиной стека: типы должны совпадать. Если стек пуст или тип не совпал, последовательность неправильная. После обработки всей строки стек должен опустеть. Алгоритм линейный по длине строки, а стек гарантирует корректное сопоставление типов скобок.

Вычисление выражения в обратной польской записи

Для записи вида “3 4 - 2 *” достаточно идти слева направо: числа помещаем в стек, операции извлекают два верхних числа, применяют оператор и возвращают результат обратно. Использование стека позволяет обрабатывать выражение за $O(n)$ без построения дерева разбора.

Очередь и амортизированный анализ

Очередь (queue) поддерживает операции `push`, `front`, `pop`. Реализация через два стека:

- ▷ стек *in* — элементы, добавленные в очередь;
- ▷ стек *out* — элементы, готовые к выдаче.

```
1 bool is_correct(const string& s) {
2     stack<char> st;
3     auto match = [](char open, char close) {
4         return (open == '(' && close == ')') ||
5                (open == '[' && close == ']') ||
6                (open == '{' && close == '}');
7     };
8     for (char c : s) {
9         if (c == '(' || c == '[' || c == '{') {
10             st.push(c);
11         } else {
12             if (st.empty() || !match(st.top(), c)) {
13                 return false;
14             }
15             st.pop();
16         }
17     }
18     return st.empty();
19 }
```

Проверка правильности скобочной последовательности

```
1 int evaluate_rpn(const vector<string>& tokens) {
2     stack<int> st;
3     for (const string& token : tokens) {
4         if (token == "+" || token == "-" || token == "*" || token == "/") {
5             int b = st.top(); st.pop();
6             int a = st.top(); st.pop();
7             if (token == "+") st.push(a + b);
8             else if (token == "-") st.push(a - b);
9             else if (token == "*") st.push(a * b);
10            else st.push(a / b);
11        } else {
12            st.push(stoi(token));
13        }
14    }
15    return st.top();
16 }
```

Вычисление выражения в обратной польской записи

```
1 struct QueueTwoStacks {
2     stack<int> in, out;
3
4     void transfer() {
5         if (!out.empty()) return;
6         while (!in.empty()) {
7             out.push(in.top());
8             in.pop();
9         }
10    }
11
12    void push(int x) {
13        in.push(x);
14    }
15
16    int front() {
17        transfer();
18        return out.top();
19    }
20
21    void pop() {
22        transfer();
23        out.pop();
24    }
25 };
```

Очередь, реализованная двумя стеками

Когда *out* пуст, переливаем все элементы из *in*; каждому элементу достаточно один раз переложиться, поэтому амортизированная стоимость операций — $O(1)$.

Функция `TRANSFER` выполняется только при опустошении S_{out} , поэтому каждый элемент перекладывается из S_{in} в S_{out} не более одного раза. Отсюда амортизированная стоимость операций `PUSH`, `FRONT` и `POP` равна $O(1)$.

Замечание. Для формального амортизированного анализа удобно ввести потенциал $\Phi = |S_{\text{in}}|$. Тогда:

- ▷ **push**: реальная цена 1, потенциал растёт на 1, амортизированная стоимость $O(1)$;
- ▷ **pop**: если S_{out} не пуст, стоимость $O(1)$ и потенциал не меняется; при вызове `transfer` потенциал падает на число переложённых элементов и «оплачивает» их;
- ▷ **front**: только читает верхушку S_{out} , поэтому итого $O(1)$.

Таким образом, даже при длинных сериях операций сложность остаётся $O(1)$ в амортизированном смысле.

Замечание. Если каждый стек хранит вместе со значением текущий минимум (например, пары `(value, stackMin)`), то очередь из двух стеков позволяет поддерживать запрос `get_min`. Минимум очереди равен минимуму из двух стеков, что полезно в задачах про скользящее окно.

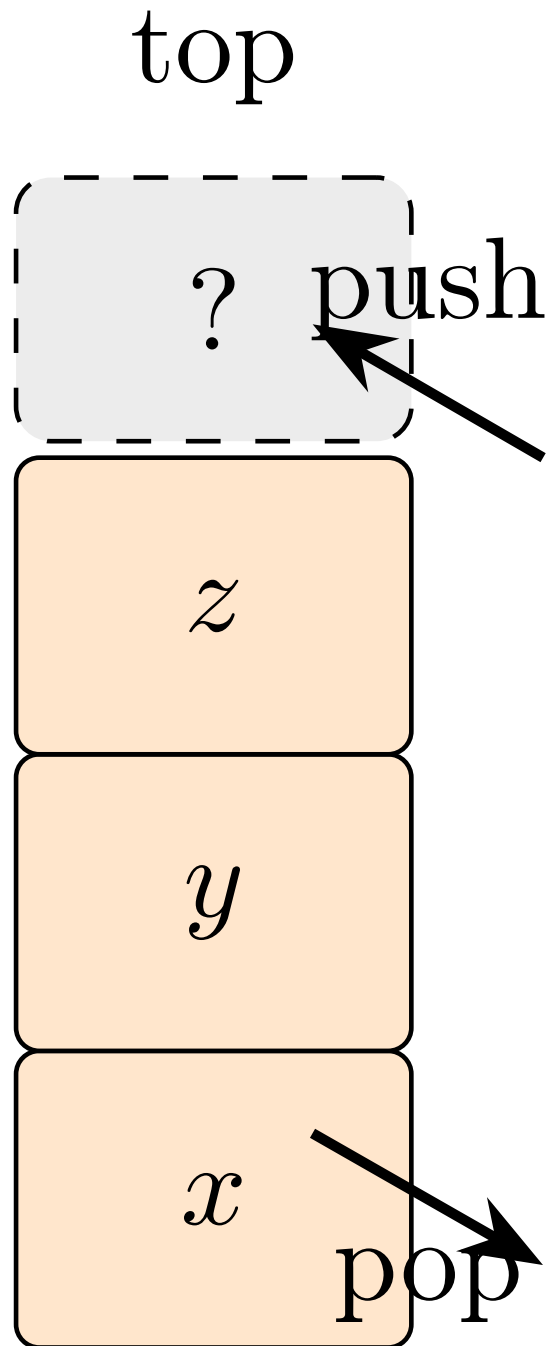


Рис. 5: Стек: операции push/pop работают с вершиной структуры.

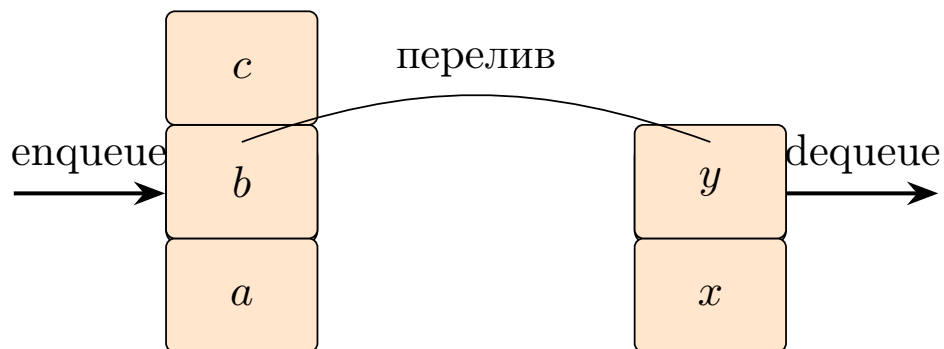


Рис. 6: Очередь через два стека: новые элементы добавляются в *in*, а при опустении *out* содержимое переливается, поэтому каждый элемент переносится не более одного раза.

Лекция 5. Кучи: расширенный функционал

Рассматриваем двоичную мин-кучу, реализованную в массиве. Основные операции: `insert`, `get_min`, `extract_min`, `decrease_key`, `erase`.

Операция `decrease_key`

Указатель `ptr` однозначно определяет позицию узла в массиве. Чтобы уменьшить ключ на Δ , достаточно вычесть значение и применить процедуру `sift_up`, пока нарушается условие кучи.

1. Пока $v > 1$ и ключ в вершине меньше ключа у родителя, обмениваем значения.
2. После обмена поднимаем указатель к родителю $v \leftarrow \lfloor v/2 \rfloor$ и повторяем проверку.
3. Как только инвариант кучи восстановлен (или достигнут корень), просеивание заканчивается.

```
1 void sift_up(vector<int>& heap, int v) {  
2     while (v > 1 && heap[v] < heap[v / 2]) {  
3         swap(heap[v], heap[v / 2]);  
4         v /= 2;  
5     }  
6 }
```

Процедура `sift_up`

Процедура восстанавливает инвариант «значение в вершине не больше значений у детей», поднимая нарушивший порядок элемент до тех пор, пока родитель снова не станет меньше либо мы не достигнем корня.

Удаление произвольного элемента (по указателю)

Чтобы удалить элемент в вершине v :

1. Выполнить `decrease_key` до значения $-\infty$.
2. Вызвать `extract_min`, что удалит элемент из корня.

Если операции выполняются по указателям, поиск элемента не требуется — это значительно упрощает задачу.

После вызова `SIFTUP` ключ оказывается в корне, откуда его безопасно удалить с помощью `EXTRACTMIN`. Если вместо указателя передано само значение, требуется дополнительная структура для поиска позиции, например хеш-таблица «ключ $\rightarrow$ список вершин».

```

1 void erase_node(vector<int>& heap, int v) {
2     const int NEG_INF = -1'000'000'000;    // value smaller than any key
3     heap[v] = NEG_INF;
4     sift_up(heap, v);
5     extract_min(heap);
6 }

```

Удаление элемента кучи по указателю

Процедура sift_down

При `extract_min` на корень перемещается последний элемент массива. Чтобы восстановить свойство мин-кучи, спускаем его вниз:

1. Пока вершина v имеет хотя бы левого сына ($2v \leq n$), выбираем из детей меньшего: $u = 2v$ или $u = 2v + 1$, если правый существует и его ключ меньше.
2. Если $a_v \leq a_u$, просеивание завершено: условие кучи соблюдено.
3. Иначе меняем местами a_v и a_u , после чего продолжаем процесс со следующей вершины $v \leftarrow u$.

```

1 void sift_down(vector<int>& heap, int v) {
2     int n = (int)heap.size() - 1;    // heap is 1-indexed
3     while (2 * v <= n) {
4         int u = 2 * v;
5         if (u + 1 <= n && heap[u + 1] < heap[u]) {
6             ++u;
7         }
8         if (heap[v] <= heap[u]) {
9             break;
10        }
11        swap(heap[v], heap[u]);
12        v = u;
13    }
14 }

```

Процедура `sift_down`

При спуске корневой элемент сравнивается с минимальным сыном; если порядок нарушен, происходит обмен и процесс повторяется. Высота дерева равна $O(\log n)$, значит и время работы `SIFTDOWN` логарифмическое.

Биномиальные кучи

Биномиальная куча состоит из коллекции биномиальных деревьев B_k степени k . В каждом B_k у корня 2^k потомков, высота равна k , а поддеревья корня — это бинарные де-

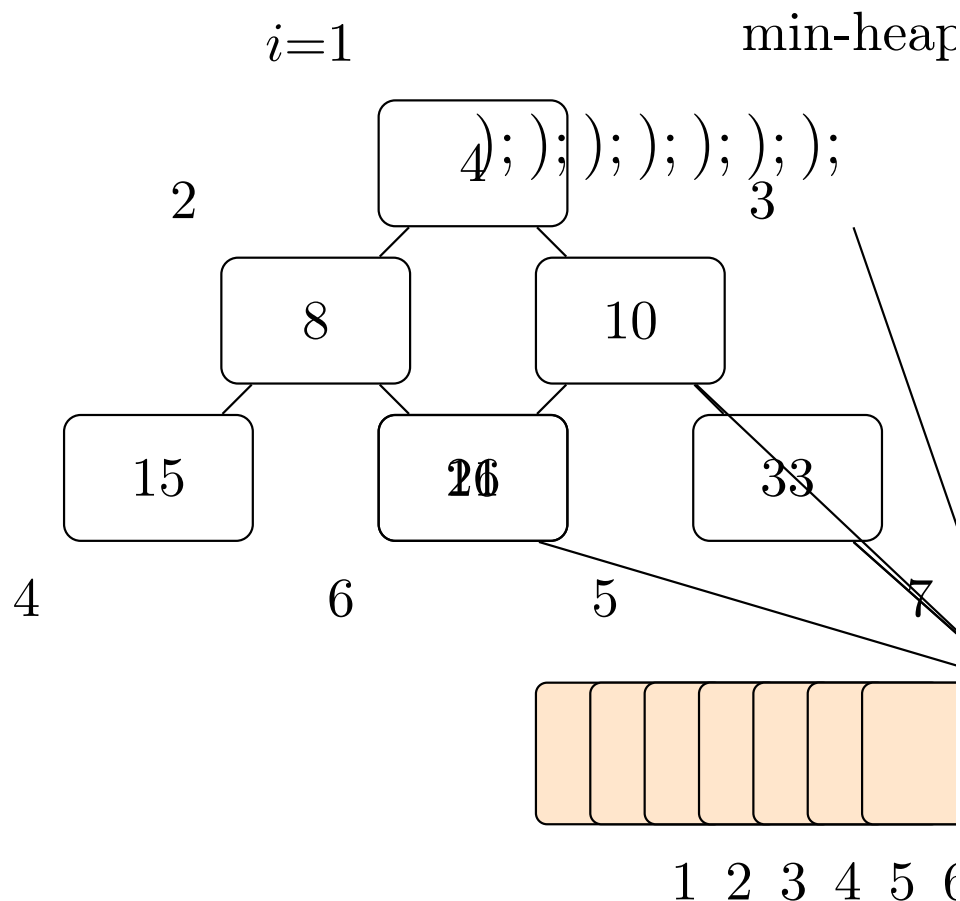


Рис. 7: Структура двоичной кучи: массив и дерево соответствуют друг другу по индексам.

ревью $B_0, B_1, \dots, B_{k-1}$. Интерфейс поддерживает операции `insert`, `get_min`, `extract_min`, `decrease_key` и `meld`.

- ▷ Корни деревьев хранятся в отсортированном по степени списке. За счёт двоичного представления размера каждая степень встречается не более одного раза.
- ▷ Слияние (`meld`) двух куч сводится к покомпонентному сложению двоичных чисел: объединяем списки корней и выполняем *перенос* одинаковых степеней путём операции `link`.
- ▷ Операции `insert` и `meld` реализуются одинаково: создаём кучу из одного элемента (дерево B_0) и сливаем.

Функция `link` прикрепляет дерево меньшего ключа корнем к дереву большего ключа, увеличивая степень получившейся вершины на единицу. После нормализации списка корней каждому разряду двоичного представления соответствует не больше одной вершины.

Операции и сложность биномиальной кучи

- ▷ `get_min`. Линейный проход по списку корней — $O(\log n)$, потому что вершин в списке не более $\lfloor \log_2 n \rfloor + 1$.

```

1 def meld(heap_a, heap_b):
2     roots = merge_by_degree(heap_a.roots, heap_b.roots)
3     prev = None
4     curr = roots
5     while curr and curr.next:
6         nxt = curr.next
7         if curr.degree != nxt.degree or \
8             (nxt.next and nxt.next.degree == curr.degree):
9             prev, curr = curr, nxt
10        else:
11            if curr.key <= nxt.key:
12                curr.next = nxt.next
13                link(nxt, curr)                # nxt becomes child of curr
14            else:
15                if prev:
16                    prev.next = nxt
17                else:
18                    roots = nxt
19                link(curr, nxt)
20                curr = nxt
21    return roots

```

Слияние списков корней биномиальных куч

- ▷ **extract_min**. Удаляем корень B_k с минимальным ключом, инвертируем порядок его детей (они уже образуют корректный набор деревьев $B_0, \dots, B_{k-1}$) и сливаем с оставшейся кучей. Асимптотика: $O(\log n)$.
- ▷ **decrease_key**. Гоним вершину вверх по дереву обменами с родителями, пока инвариант не восстановится. Глубина дерева не превосходит $\log n$.
- ▷ **meld**. Одна проходка по спискам корней — $O(\log n)$.

Каждая операция укладывается в $O(\log n)$ благодаря ограниченному числу биномиальных деревьев.

Фибоначчиевы кучи

Фибоначчиева куча ослабляет требования к биномиальной: вместо жёсткого множества деревьев поддерживается список корней произвольных степеней. Куча хранит:

- ▷ указатель на минимальный элемент;
- ▷ двусвязный список корней;
- ▷ для каждой вершины — указатель на родителя, список детей, степень и *флаг маркировки* (отмечаем узел, если он потерял ребёнка).

```

1 def extract_min(heap):
2     z = heap.min
3     if z is None:
4         return None
5     for child in iterate_children(z):
6         add_root(heap, child)
7         child.parent = None
8     remove_from_root_list(heap, z)
9     if heap.root_list is None:
10        heap.min = None
11    else:
12        heap.min = heap.root_list
13        consolidate(heap)      # merge roots of equal degree
14    heap.n -= 1
15    return z.key

```

Удаление минимума во фибоначчиевой куче

Ключевая идея — откладывать дорогостоящую нормализацию до вызова `extract_min`. Операции `insert` и `meld` добавляют новый корень в список за $O(1)$.

Процедура `consolidate` использует массив размером $\lfloor \log_{\varphi} n \rfloor$ для связывания корней одинаковой степени. Связывание проводится операцией `link`, как и в биномиальной куче, но только над корнями.

Уменьшение ключа и «каскадные» обрезки

```

1 def decrease_key(heap, x, delta):
2     x.key -= delta
3     y = x.parent
4     if y and x.key < y.key:
5         cut(heap, x, y)
6         cascading_cut(heap, y)
7     if x.key < heap.min.key:
8         heap.min = x

```

Уменьшение ключа во фибоначчиевой куче

Если вершина теряет второго ребёнка, она отмечается и при следующем снижении ключа у потомка сама вырезается в список корней. Это гарантирует, что степень каждой вершины $O(\log n)$.

Амортизированный анализ фибоначчиевой кучи

Используем потенциал $\Phi = T + 2M$, где T — число деревьев в списке корней, M — количество помеченных вершин.

- ▷ `insert` и `meld`: $\Delta\Phi = 1$, фактическая стоимость $O(1)$, амортизированная — $O(1)$.
- ▷ `decrease_key`: каждая операция `cut` уменьшает M , компенсируя реальную работу; амортизированная цена $O(1)$.
- ▷ `extract_min`: реальная цена $O(\log n)$ (работа `consolidate`), при этом потенциал не увеличивается больше, чем на $O(\log n)$.

Итог: `insert`, `meld`, `decrease_key` имеют амортизированное время $O(1)$, а `extract_min` выполняется за $O(\log n)$. Это улучшает стоимость операций по сравнению с биномиальными кучами и важно для алгоритмов Дейкстры и Прима.

Стек с минимумом

Чтобы поддерживать операции `push`, `pop`, `top` и `get_min` за $O(1)$, достаточно хранить вместе со значением текущий минимум на префиксе стека. Каждая вершина хранит ин-

```

1 struct MinStack {
2     vector<pair<int, int>> data;           // (value, current minimum)
3
4     void push(int x) {
5         int curMin = data.empty() ? x : min(x, data.back().second);
6         data.emplace_back(x, curMin);
7     }
8
9     void pop() {
10        data.pop_back();
11    }
12
13    int top() const {
14        return data.back().first;
15    }
16
17    int get_min() const {
18        return data.back().second;
19    }
20
21    bool empty() const {
22        return data.empty();
23    }
24 };

```

Стек с поддержкой минимума

формацию о минимуме «под собой», поэтому при удалении вершины весь стек остаётся корректным.

Очередь с минимумом

Соберём очередь из двух стеков *in* и *out*. В стек *in* кладём новые элементы, из *out* извлекаем голову очереди. Как только *out* пуст, переносим туда все элементы из *in*, меняя порядок на обратный. Минимум очереди равен минимуму из двух стеков, поэтому

```
1 struct MinQueue {
2     MinStack in, out;
3
4     void push(int x) {
5         in.push(x);
6     }
7
8     void transfer() {
9         if (!out.empty()) return;
10        while (!in.empty()) {
11            int v = in.top();
12            in.pop();
13            out.push(v);
14        }
15    }
16
17    void pop() {
18        transfer();
19        out.pop();
20    }
21
22    int front() {
23        transfer();
24        return out.top();
25    }
26
27    int get_min() {
28        if (in.empty()) return out.get_min();
29        if (out.empty()) return in.get_min();
30        return min(in.get_min(), out.get_min());
31    }
32};
```

Очередь с минимумом через два стека

операция `get_min` выполняется за $O(1)$.

Амортизированный анализ очереди с минимумом

Каждый элемент добавляется в очередь один раз, один раз переносится из *in* в *out* и затем удаляется. Следовательно, при q операциях суммарная стоимость переключений составляет $O(q)$, а амортизированная цена `push`, `pop`, `front` и `get_min` равна $O(1)$.

Лекция 6. Удаление из кучи

Новая операция `erase(x)`

Хотим уметь удалять произвольный элемент x из кучи. Возможны две постановки:

- ▷ **Удаление по указателю.** Нам прямо дают позицию вершины v . В этом случае используем приём из предыдущей лекции: уменьшаем значение до $-\infty$ и вызываем `extract_min`.
- ▷ **Удаление по значению.** Нам сообщают ключ x , но не место, где он хранится. Для чистой двоичной кучи это сложнее: требуется поддерживать дополнительную структуру (например, хеш-таблицу «значение $\rightarrow$ список указателей»). В рамках текущей модели сосредотачиваемся на удалении по указателю.

```
1 struct Heap {  
2     vector<int> a;                // data stored from index 1; a[0] is  
3     unused  
4  
5     void erase(int v) {  
6         const int NEG_INF = std::numeric_limits<int>::min();  
7         a[v] = NEG_INF;  
8         sift_up(v);                // bubble element to the root  
9         extract_min();             // and remove the root  
10    }  
};
```

Удаление элемента из кучи по указателю

Удаление по значению реализуют через дополнительную структуру «значение $\rightarrow$ список индексов». При каждом обмене в куче необходимо поддерживать и этот список, чтобы указатели оставались корректными.

Практические замечания

- ▷ В задачах на обработку событий (симуляция, физические модели) часто требуется отменять отложенные события. При появлении новой информации событие становится недействительным, и его нужно удалить из приоритетной очереди — здесь необходима операция `erase`.
- ▷ Если вместо указателя передаётся идентификатор (индекс), необходимо следить за его актуальностью при перестановках элементов внутри массива кучи.
- ▷ Процедуры `sift_up` и `sift_down` остаются базовыми кирпичиками: каждая операция кучи сводится к их последовательному применению.

Биномиальные кучи

Чтобы поддерживать слияние, переходим к структуре, где куча представляется набором биномиальных деревьев. Биномиальное дерево B_k :

- ▷ содержит 2^k вершин и имеет высоту k ;
- ▷ у корня есть дети $B_{k-1}, B_{k-2}, \dots, B_0$;
- ▷ образуется из B_{k-1} добавлением нового корня, к которому прикрепляются дети B_{k-1} .

Храним массив корней `root[k]` — указатель на корень дерева B_k (если такого дерева нет, `root[k] = nullptr`). Множество корней соответствует двоичному представлению количества элементов.

Операции

- ▷ `get_min`: достаточно пройти по массиву корней и выбрать минимальный ключ. В массиве не более $\lfloor \log_2 n \rfloor + 1$ элементов, значит операция занимает $O(\log n)$.
- ▷ `merge`: аналог сложения двоичных чисел. Если в разряде k оказалось два дерева, объединяем их в дерево B_{k+1} с помощью `link`. Если три дерева, одно оставляем, два объединяем и переносим «перенос» в следующий разряд.

```

1 def merge_roots(roots_a, roots_b):
2     carry = None
3     result = [None] * MAX_K
4     for k in range(MAX_K):
5         candidates = [t for t in (roots_a[k], roots_b[k], carry) if t]
6         if len(candidates) == 0:
7             result[k] = None
8             carry = None
9         elif len(candidates) == 1:
10            result[k] = candidates[0]
11            carry = None
12        elif len(candidates) == 2:
13            carry = link(candidates[0], candidates[1]) # produce B_{k+1}
14            result[k] = None
15        else:
16            result[k] = candidates[0]
17            carry = link(candidates[1], candidates[2])
18    return result

```

Слияние корней биномиальных куч

Сложность

- ▷ **insert**: создаём кучу из одного элемента (дерево B_0) и выполняем **merge** — $O(\log n)$.
- ▷ **extract_min**: находим минимальный корень, удаляем его, инвертируем порядок его поддеревьев $B_{k-1}, \dots, B_0$ и сливаем с оставшейся кучей — $O(\log n)$.
- ▷ **decrease_key**: как в двоичной куче, спускаемся вверх обменами. Высота дерева не превосходит $\log n$.

Фибоначчиевы кучи

Фибоначчиева куча улучшает амортизированные оценки: **insert**, **merge**, **decrease_key** выполняются за $O(1)$ амортизированно, **extract_min** — за $O(\log n)$. Потенциал $\Phi = T + 2M$, где T — число деревьев, M — число помеченных вершин.

- ▷ **insert**, **merge**: увеличивают потенциал на 1, фактическая цена — $O(1)$.
- ▷ **decrease_key**: каскадные обрезки уменьшают Φ , поэтому амортизированная стоимость $O(1)$.
- ▷ **extract_min**: требует $O(\log n)$ реального времени; рост потенциала также ограничен $O(\log n)$.

Амортизированное время

Для серии из q операций в биномиальной куче суммарное время $O(q \log n)$; в фибоначчиевой куче линейные операции имеют амортизированную цену $O(1)$, а **extract_min** остаётся $O(\log n)$.